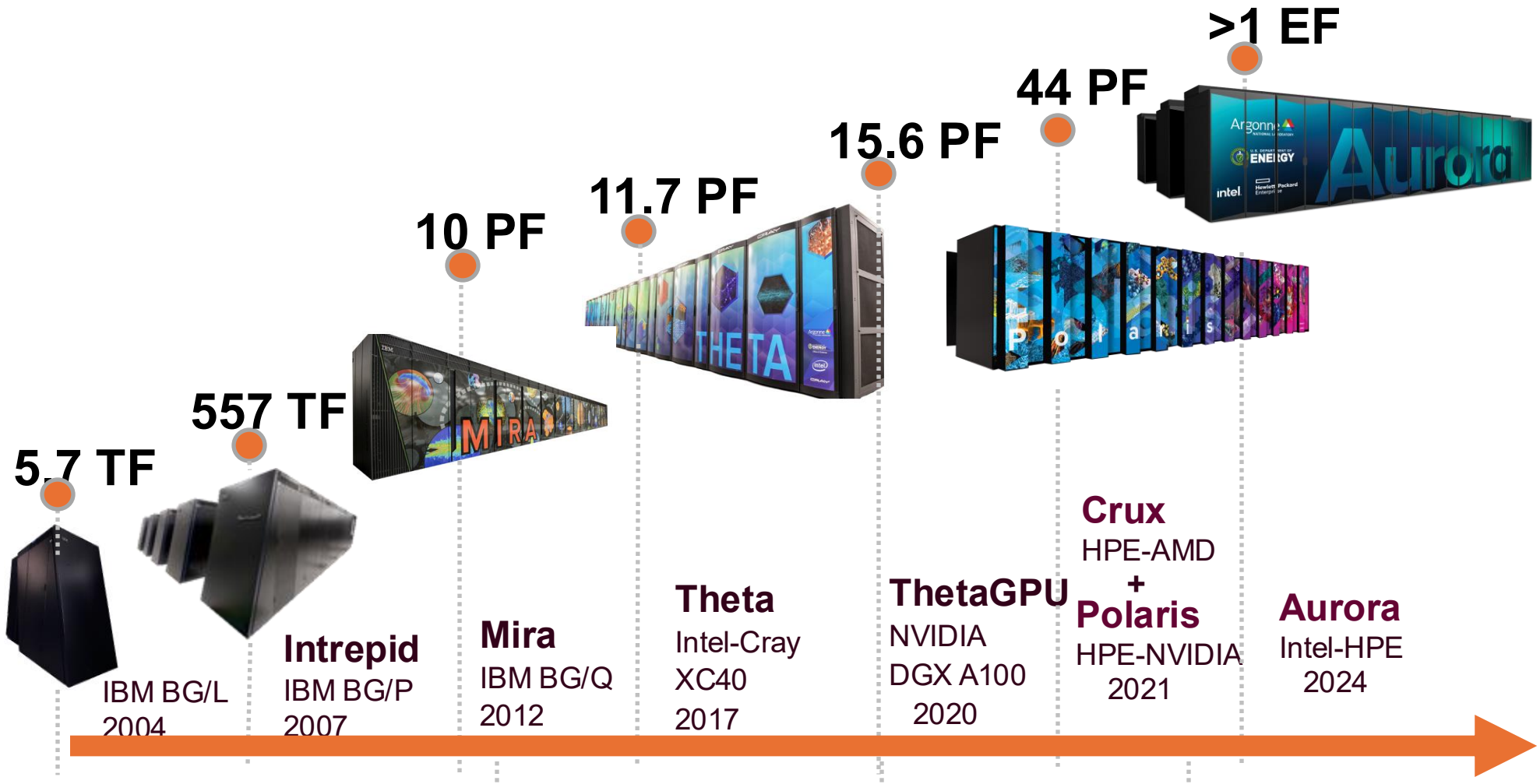
# ALCF AI Testbed

08/04/2026

Varuni Sastry

ALCF

# ALCF Systems Evolution



# ALCF AI Testbed

- Infrastructure of next-generation machines with hardware accelerators customized for artificial intelligence (AI) applications.
- Provide a platform to evaluate usability and performance of machine learning based HPC applications running on these accelerators.
- The goal is to better understand how to integrate AI accelerators with ALCF's existing and upcoming supercomputers to accelerate science insights

# ALCF AI Testbed Deployments


**Cerebras (CS-3)**
4x CS-3 nodes

**Supports:**
AI training and inference of massive AI models, CSL-SDK

**Status:**
Upgraded software stack. In production for pre-training and finetuning.


**Metis**
SambaNova SN-40L nodes

**Supports:**
AI Inference for models developed by Argonne researchers.

**Status:**
4x nodes being integrated with ALCF Inference service. Adding 2 nodes for a total of 6x.


**Tenstorrent**
2x Galaxy Nodes (64 wormhole cards)

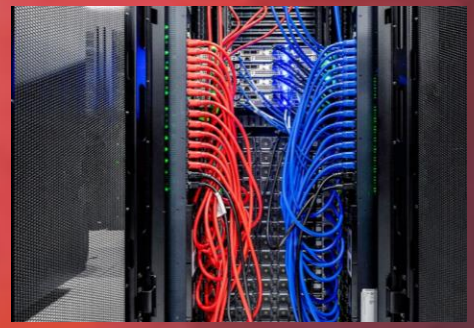
**Supports:**
AI Inference (one node) and SDK development (one node).

**Status:**
Two nodes configured and available to Argonne users. Will be made available to ALCF users.


**GroqRack**
72 accelerators (9 nodes x 8 accelerators per node)

**Supports:**
AI inference using Language Processing Unit (LPU) technology.

**Status:**
Available for allocation requests.



**Graphcore Bow Pod64**
64 accelerators (4 nodes x 16 accelerators per node)

**Supports:**
ML and AI workloads.

**Status:**
Available for allocation requests.

# ALCF Documentation

**https://docs.alcf.anl.gov/ai-testbed/getting-started/**







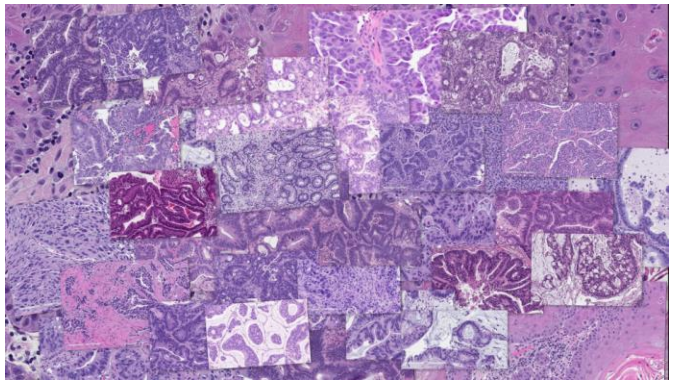
## ALCF AI Testbed


The ALCF AI Testbed houses some of the most advanced AI accelerators for scientific research.
The goal of the testbed is to enable explorations into next-generation machine learning applications and workloads, enabling the ALCF and its user community to help define the role of AI accelerators in scientific computing and how to best integrate such technologies with supercomputing resources.
The AI accelerators complement the ALCF's current and next-generation supercomputers to provide a state-of-the-art computing environment that supports pioneering research at the intersection of AI, big data, and high performance computing (HPC).
The platforms are equipped with architectural features that support AI and data-centric workloads, making them well suited for research tasks involving the growing deluge of scientific data produced by powerful tools, such as supercomputers, light sources, telescopes, particle accelerators, and sensors. In addition, the testbed will allow researchers to explore novel workflows that combine AI methods with simulation and experimental science to accelerate the pace of discovery.



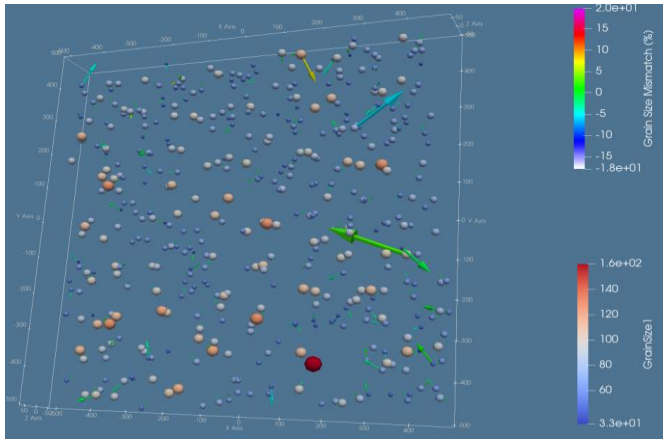
**Does this doc need an update?**
**Open an Issue on GitHub**

# AI for Science and HPC applications on AI Testbed



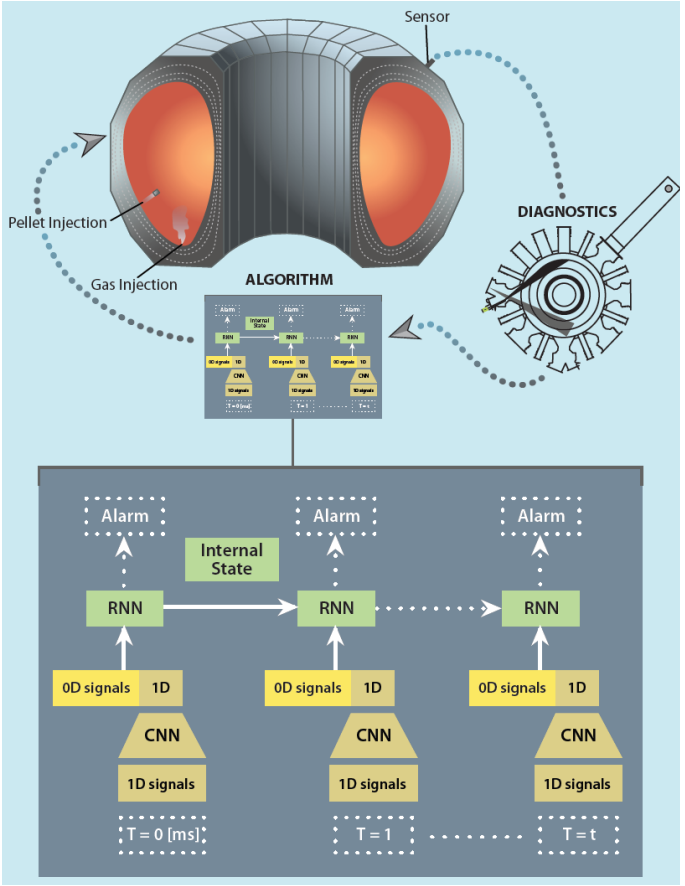
Cancer drug response prediction

(Credit: Candle)



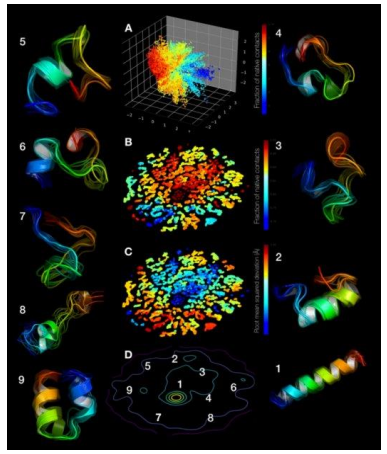
Imaging Sciences-Braggs Peak

(Credit: Z. Liu)

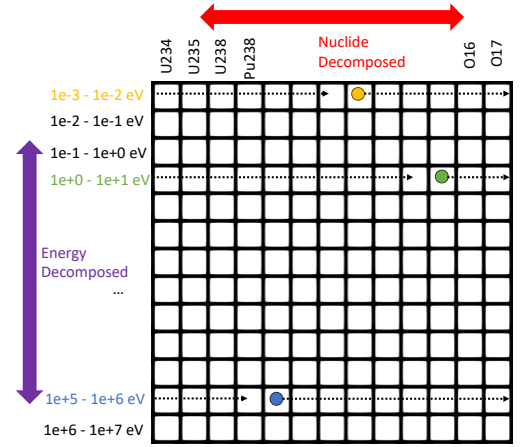



Tokomak Fusion Reactor operations

(Credit: K. Felker)



Protein-folding (Image: NCI)

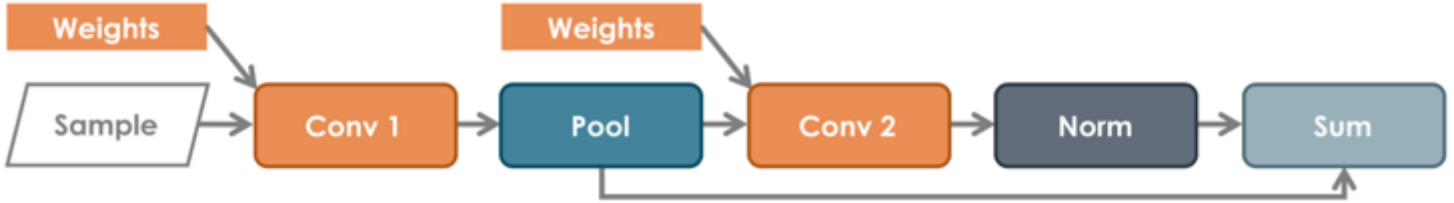



Monte Carlo Particle Transport for Reactor Simulation (Credit: J. Tramm)

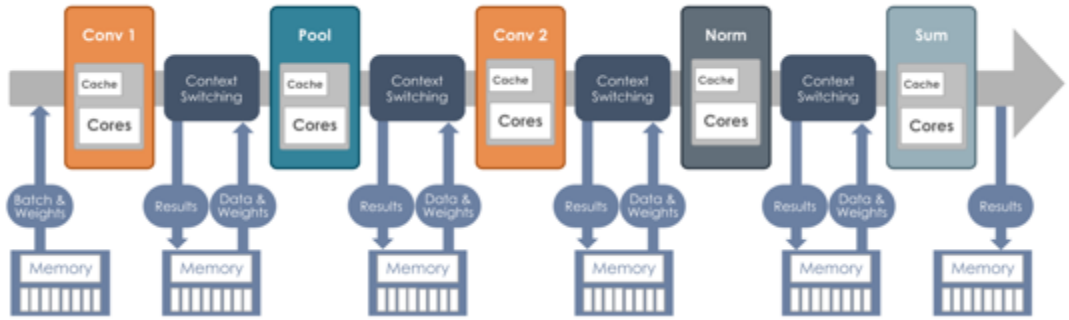
**and more..**

extremecomputingtraining.anl.gov

# Dataflow architecture

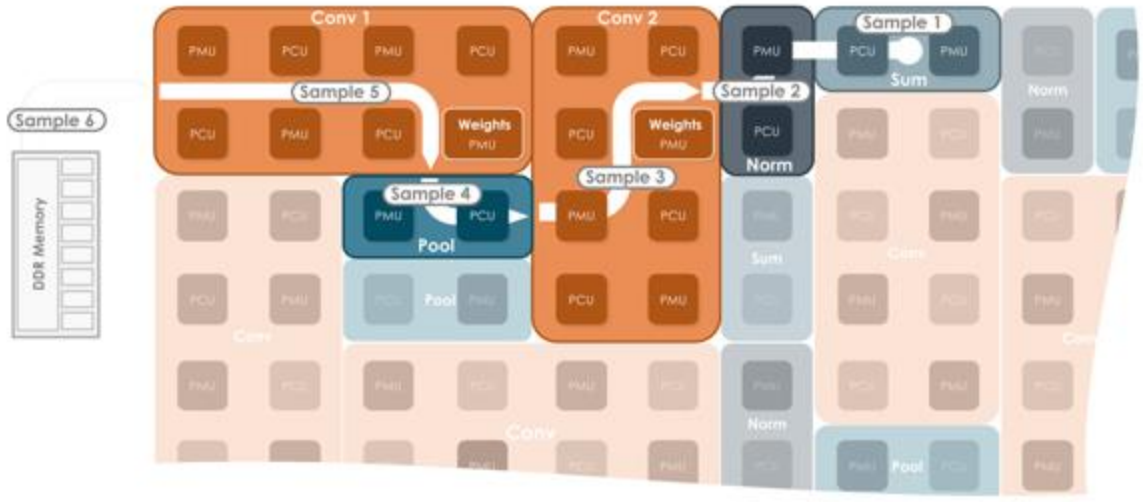



Simple Convolution Graph




The GPU way: kernel-by-kernel

Bottlenecked by memory bandwidth and host overhead




The Dataflow way: Spatial

Eliminates memory traffic and overhead

Image Courtesy: SambaNova

# Dataflow architecture

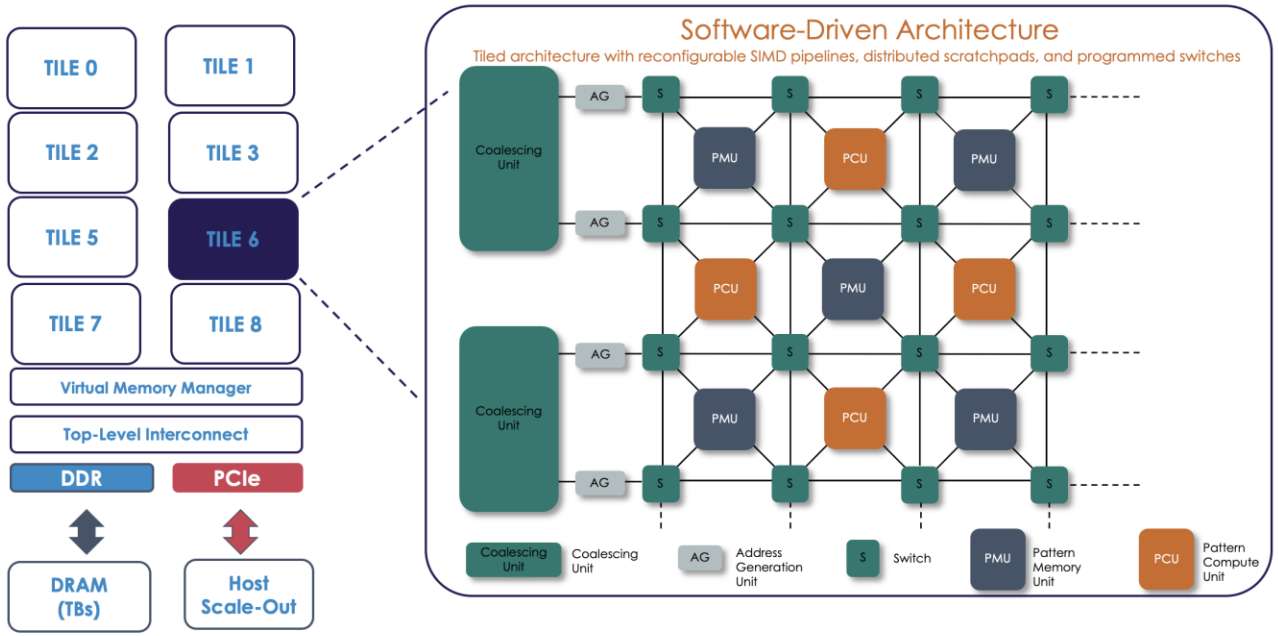
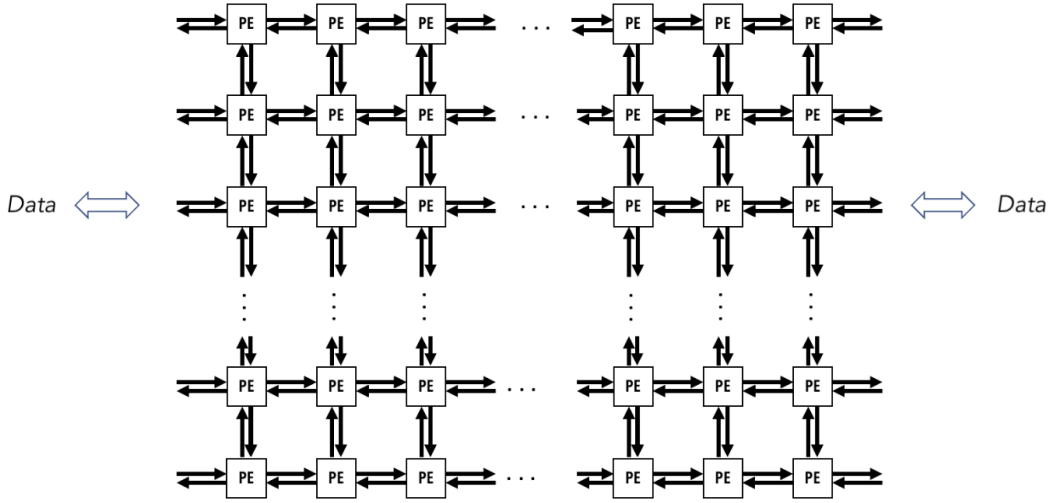



Image Courtesy: SambaNova



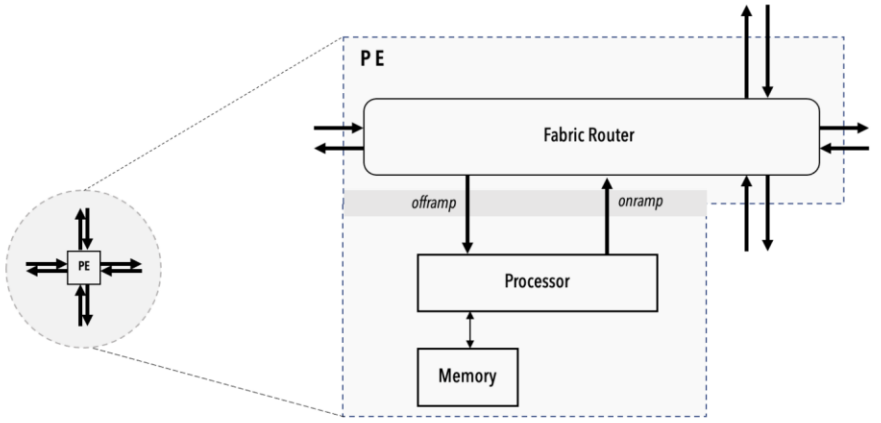
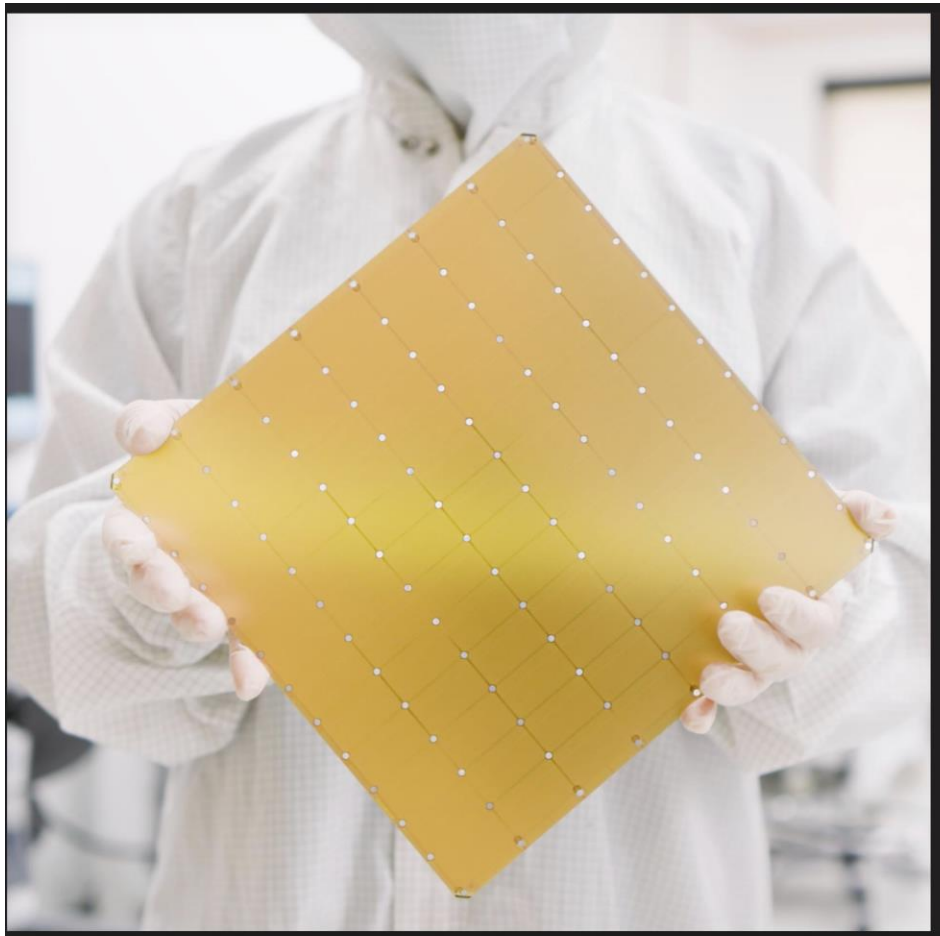




Image coutesy: SambaNova

- Interleaving of compute and memory units
- Routing data through the compute elements

# Cerebras



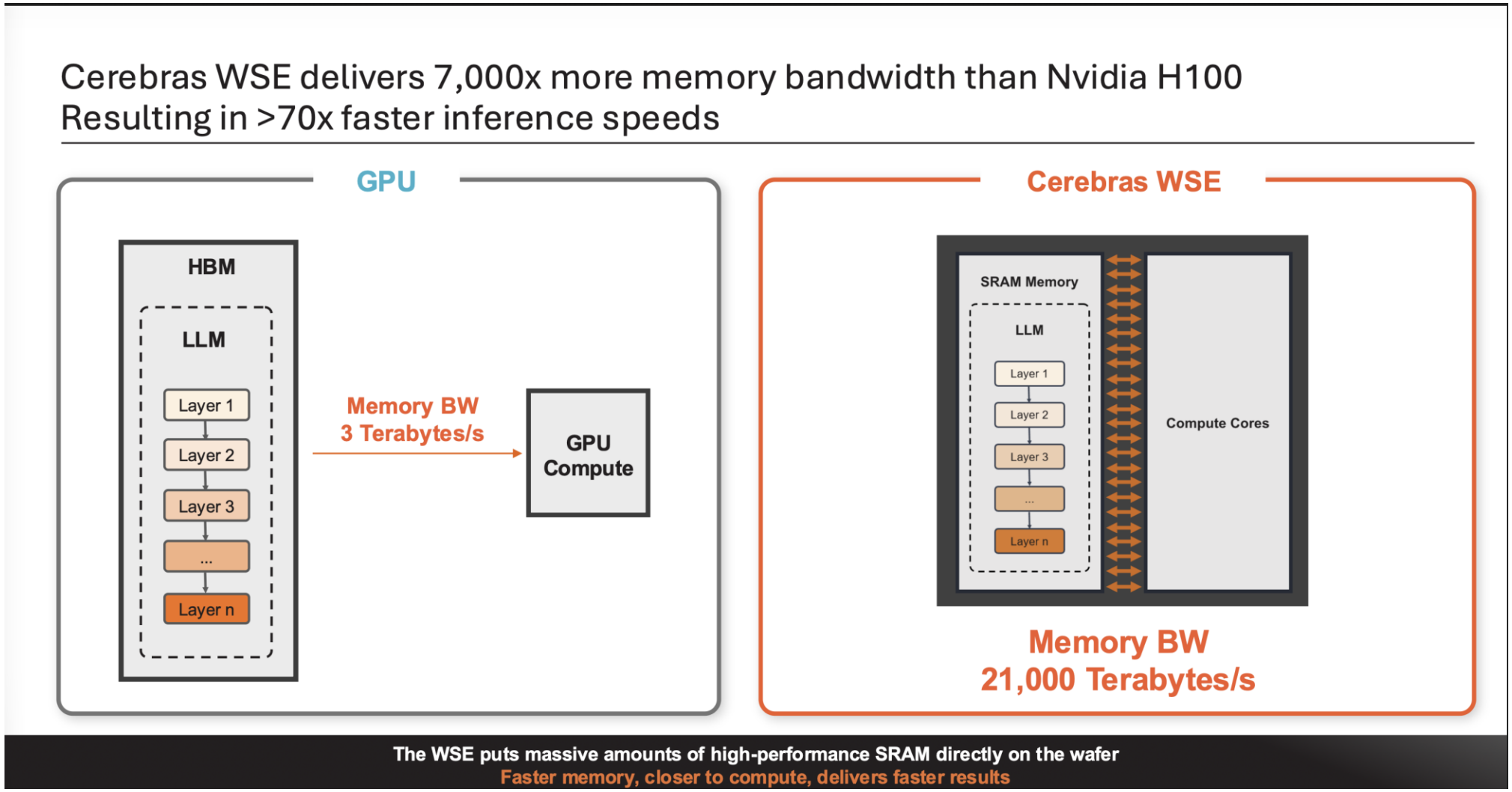
## Cerebras Wafer Scale Engine 3 (WSE-3)

**The fastest AI chip on earth again**

- 4 trillion transistors
- 46,225 mm2 silicon
- 900,000 cores optimized for sparse linear algebra
- 5nm TSMC process
- 125 Petaflops of AI compute
- 44 Gigabytes of on-chip memory
- 21 PByte/s memory bandwidth
- 214 Pbit/s fabric bandwidth

Courtesy: Cerebras

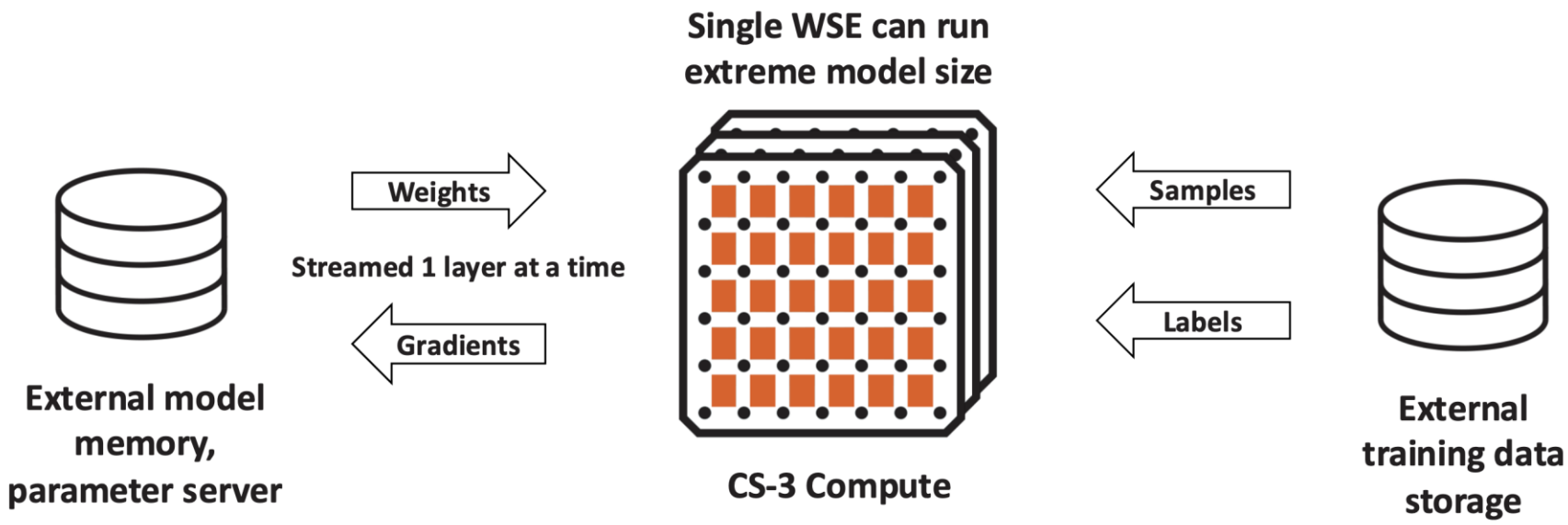
# Cerebras


Cerebras WSE delivers 7,000x more memory bandwidth than Nvidia H100
Resulting in >70x faster inference speeds
GPU
HBM
LLM
Layer 1
Layer 2
Layer 3
...
Layer n
Memory BW
3 Terabytes/s
GPU
Compute
Cerebras WSE
SRAM Memory
LLM
Layer 1
Layer 2
Layer 3
...
Layer n
Compute Cores
Memory BW
21,000 Terabytes/s
The WSE puts massive amounts of high-performance SRAM directly on the wafer
Faster memory, closer to compute, delivers faster results

# Cerebras

## Weight Streaming Technology

### Disaggregates Memory and Compute for Trillion Parameter Model Training



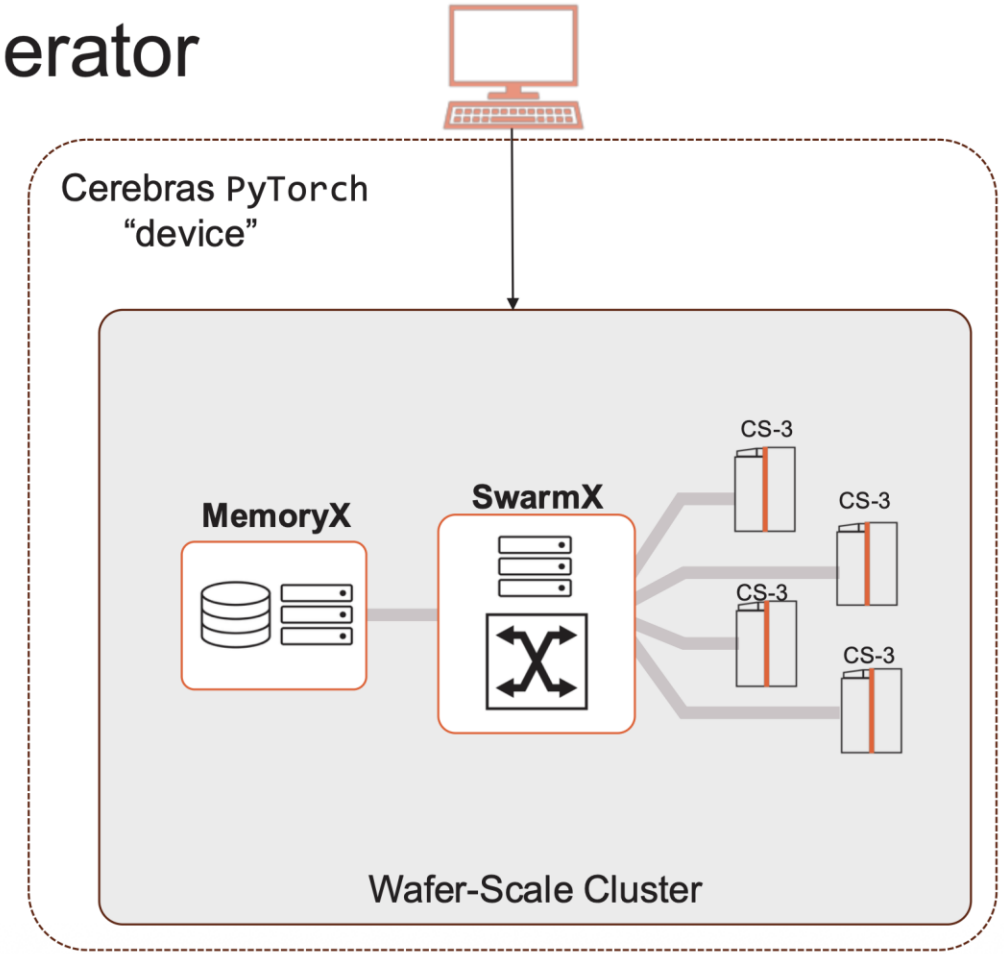

**Scale model size and training speed independently**

# Cerebras

## The Cluster *is* the ML Accelerator

**Disaggregation of memory and compute**

- Stores model weights off-wafer with fast streaming
- MemoryX size and compute device count can be scaled independently
- The entire cluster acts one remote accelerator
- All the complexity of distributed training, data parallelism, and scaling are handled automatically by the Wafer Scale Cluster
- Exposed to the user as a single PyTorch device!



# The Cerebras Modelzoo

github.com/Cerebras/modelzoo

| | |
|---|---|
| As a **starting** point | Reference model implementations:<br>GPT-3, LLaMa, Mistral, BERT, LLaVa, and much more! |
| | Data preparation scripts |
| | Configurations for multiple model scales |
| For **benchmarking** | Tuned implementations for optimal performance |
| To **develop** new models | Cerebras PyTorch & Layer APIs |
| | File organization as a template to start your own models |

# Custom Training Loops with the `cstorch` API

- Scale out to multiple CS-3s with a one-argument configuration change
- Near-linear scaling happens automatically
- We don't need to change…
  - Model code
  - Training loop
  - Effective batch size
- It's easy to supply the value programmatically for edit-free scaling

Scale out

```python
import torch
import cerebras_pytorch as cstorch

model: torch.nn.Module = Model()
model = cstorch.compile(model, "CSX")
loss_fn = torch.nn.NLLLoss()
optimizer = cstorch.optim.SGD(model.parameters(), lr=0.01)
dataloader = cstorch.utils.data.DataLoader(get_train_dataloader)
executor = cstorch.utils.data.DataExecutor(
    dataloader, cs_config=cstorch.utils.CSConfig(num_csx=16)
)

@cstorch.trace
def training_step(inputs, targets):
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = loss_fn(outputs, targets)
    loss.backward()
    optimizer.step()
    return loss

@cstorch.step_closure
def print_loss(loss: torch.Tensor):
    print(f"Loss: {loss.item()}")
```

# Scaling from 8B model to 70B

**Define the model**

- Same code as for Llama-3 8B
- Different yaml parameterization
- 70B model code is still for a *single* device

params_llama3_70b.yaml

```
### LLaMA-3 70B

hidden_size: 8192
num_hidden_layers: 80
num_heads: 64
```

**Train the model**

- Point to the upscaled model parameters
- The rest is the same as with Llama-3 8B
- Run!

training:

```
cszoo fit params_llama3_70b.yaml \
--num_csx 1
```

Courtesy: Cerebras

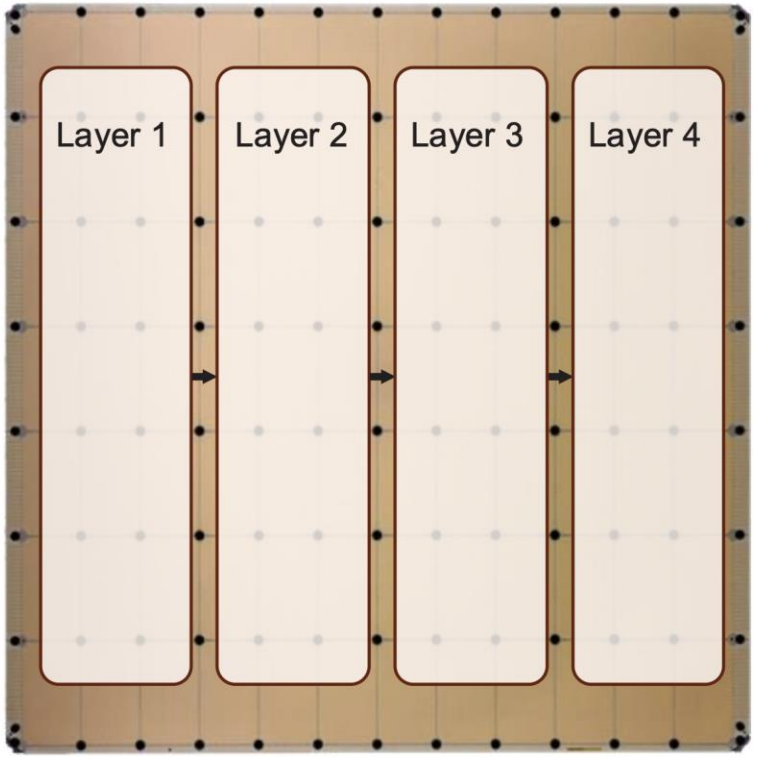
# Pipeline execution on a single chip

**Massive memory bandwidth enables opposite execution model**

- GPU: multiple chips to run a single layer
- Cerebras: fraction of a chip to run a single layer

**Pipeline mapping of model**

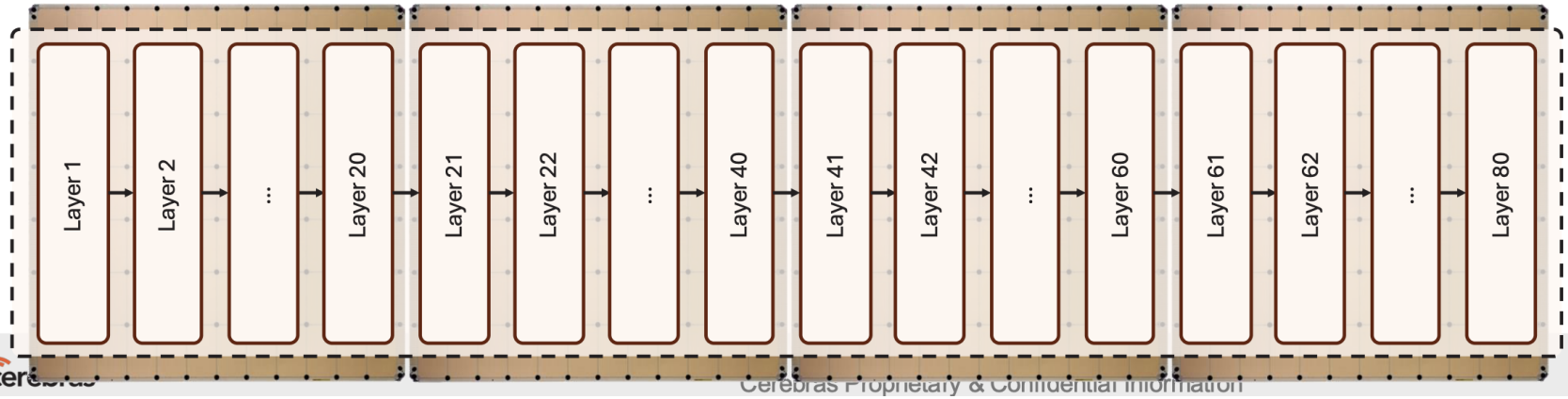
- Model layers are mapped to wafer regions
- Size of wafer region determined based on memory and compute requirements
- Model weights and KV cache stored locally in the region memory close to the compute




Courtesy: Cerebras

# Cerebras

- Models that require more memory are mapped to multiple wafers
- Keep almost all high communication on wafer, using internal high bandwidth fabric
- Transfer only activations between wafers, requiring relatively lower bandwidth
- Using CS-3 low latency RDMA over Ethernet interconnect between systems




**Llama3.1-70B**
70 billion parameters
140 GB of memory (FP16)

**4x WSE-3**
176GB of SRAM

# Cerebras CSL

https://docs.alcf.anl.gov/ai-testbed/cerebras/csl/

- Cerebras CSL (Cerebras System Language) is a low-level kernel programming language designed for the Cerebras system.
  - includes libraries for a variety of commonly used primitive operations, such as broadcasting, gathering, and scattering data across rows or columns of PEs.

- It enables users to write code that runs on individual Processing Elements (PEs) and to define the placement of programs and the routing of data on the Wafer-Scale Engine (WSE).
- 2 components: Device-code: runs directly on the wafer, host-code to facilitate data movement and execute functions on the wafer.
- Simulator mode: runs locally or Appliance mode: runs directly on CS-3.

# Metis – SN40L Inference cluster

SambaNova SN40L


- 6x SN40L nodes each with 16x SN40L Accelerators
- 1.5TB per RDU – 48TB in aggregate
- Highly optimized for inference

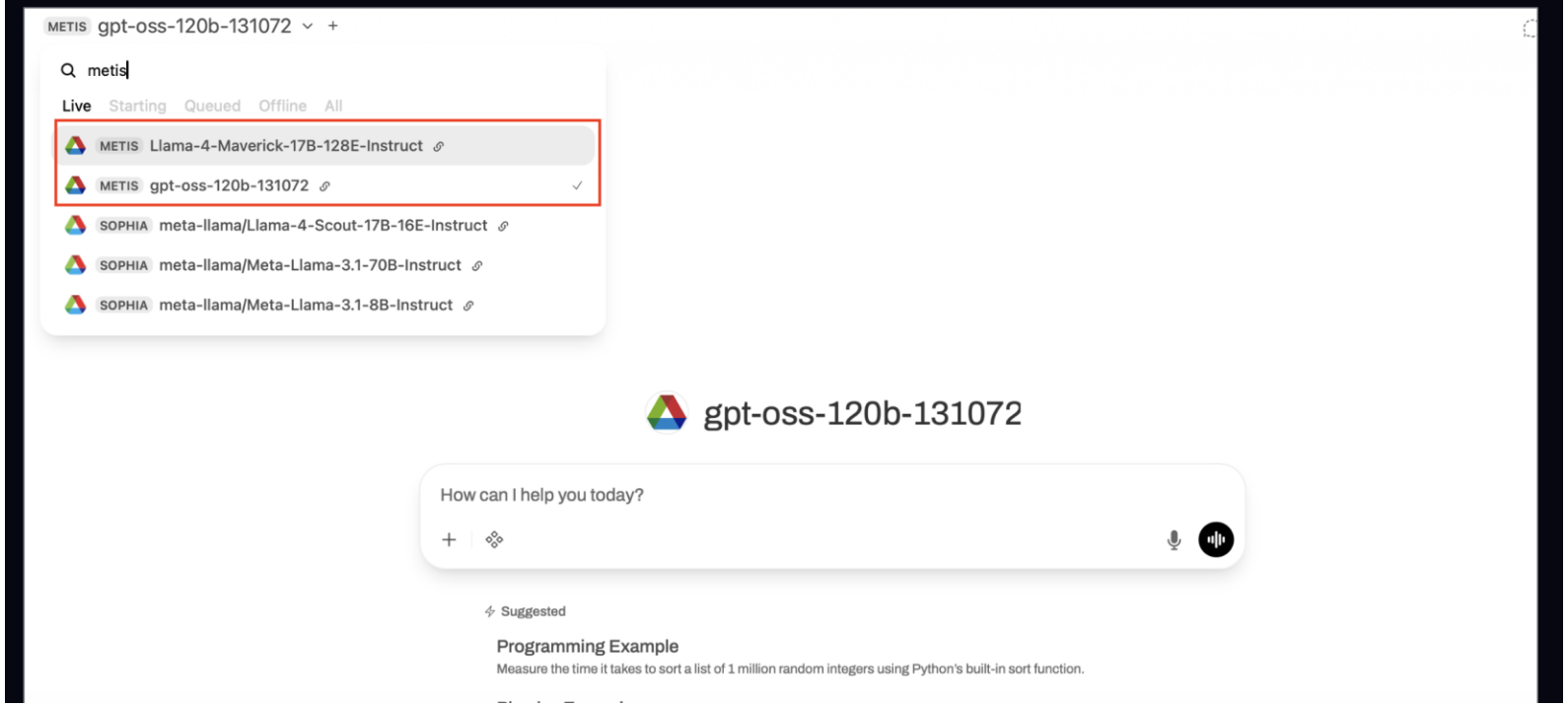
# Metis – SN40L

## Accessing the endpoints using the Web UI.

The easiest way to get started is through the web interface, accessible at https://inference.alcf.anl.gov/
The UI is based on the popular Open WebUI platform. After logging in with your ANL or ALCF credentials, you can:

1. Select a model from the dropdown menu at the top of the screen.
2. Start a conversation directly in the chat interface.

In the model selection dropdown, you can see the status of each model:



# Sambanova - SN40L

Choose sidebar display

sambanova

## SN40L: SambaNova's 4th Gen AI Chip

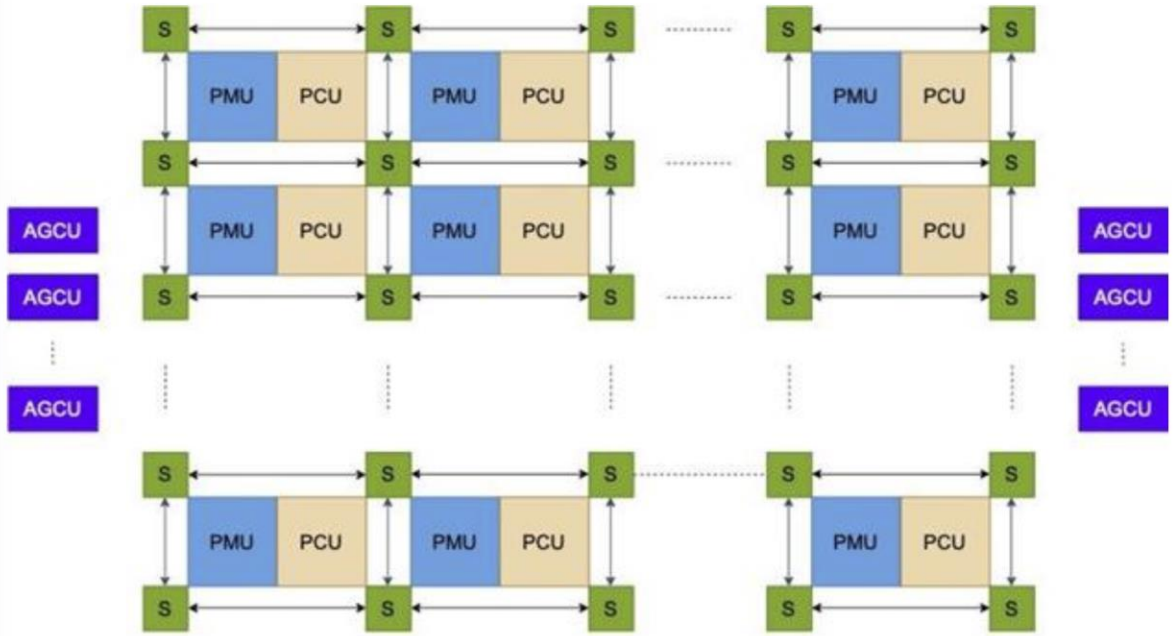
**Reconfigurable Dataflow Unit (RDU)**
**Native multi-tenancy support with fast model switching**
**Ideal for production inference, multi-tenancy, agentic workflows**


- **3-tier Dataflow Memory**
- **520 MB On-Chip SRAM Memory** → **Very fast memory for high speed inference with caching**
- **64 GB High Bandwidth Memory** → **Switch between models in as little as 2 milliseconds**
- **768 GB High Capacity DDR Memory** → **Hold large number of models in memory**

# Sambanova - SN40L

sambanova

## SN40L: Tile Architecture




1040 PCUs and PMUs

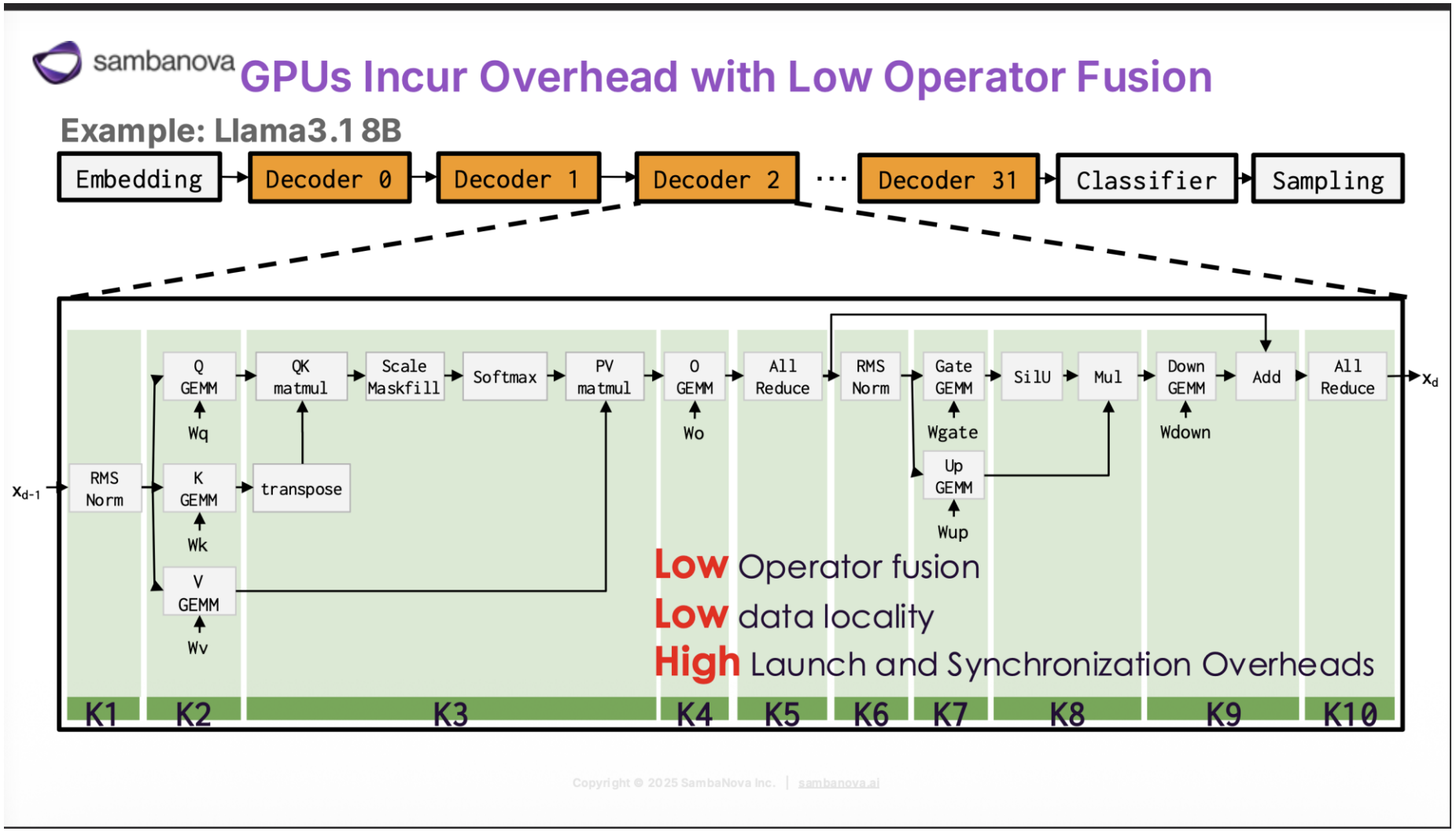
PCU: Compute unit

PMU: Memory unit

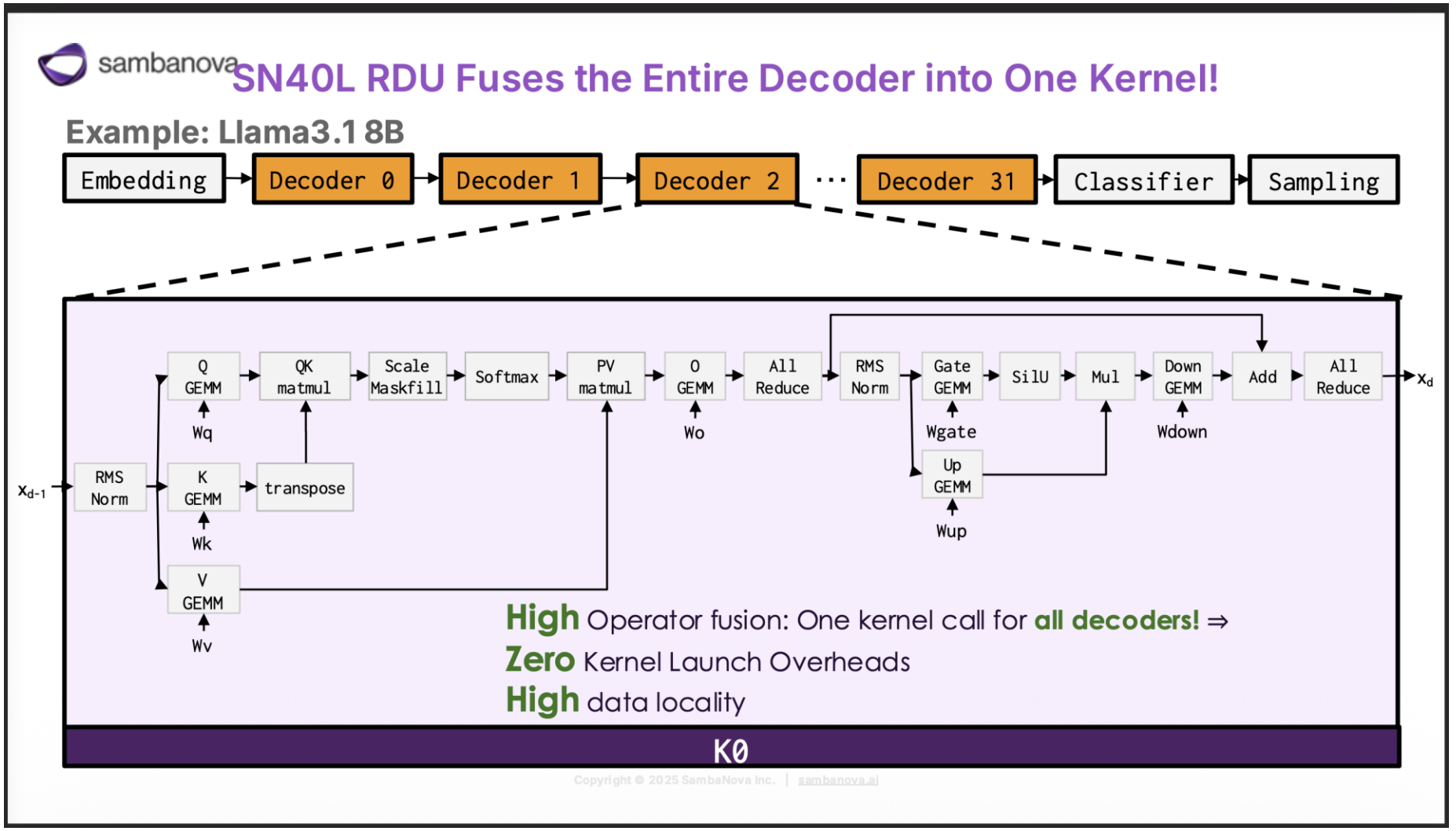
S: Mesh switches

AGCU: Portal to off-chip memory and IO

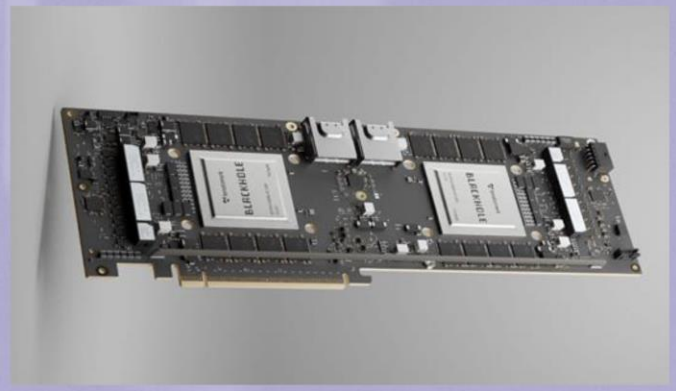
# Sambanova - SN40L

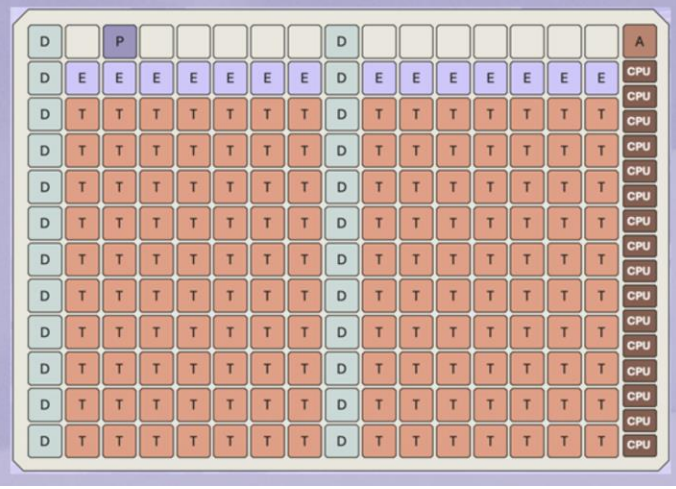


# Sambanova - SN40L



# Tenstorrent





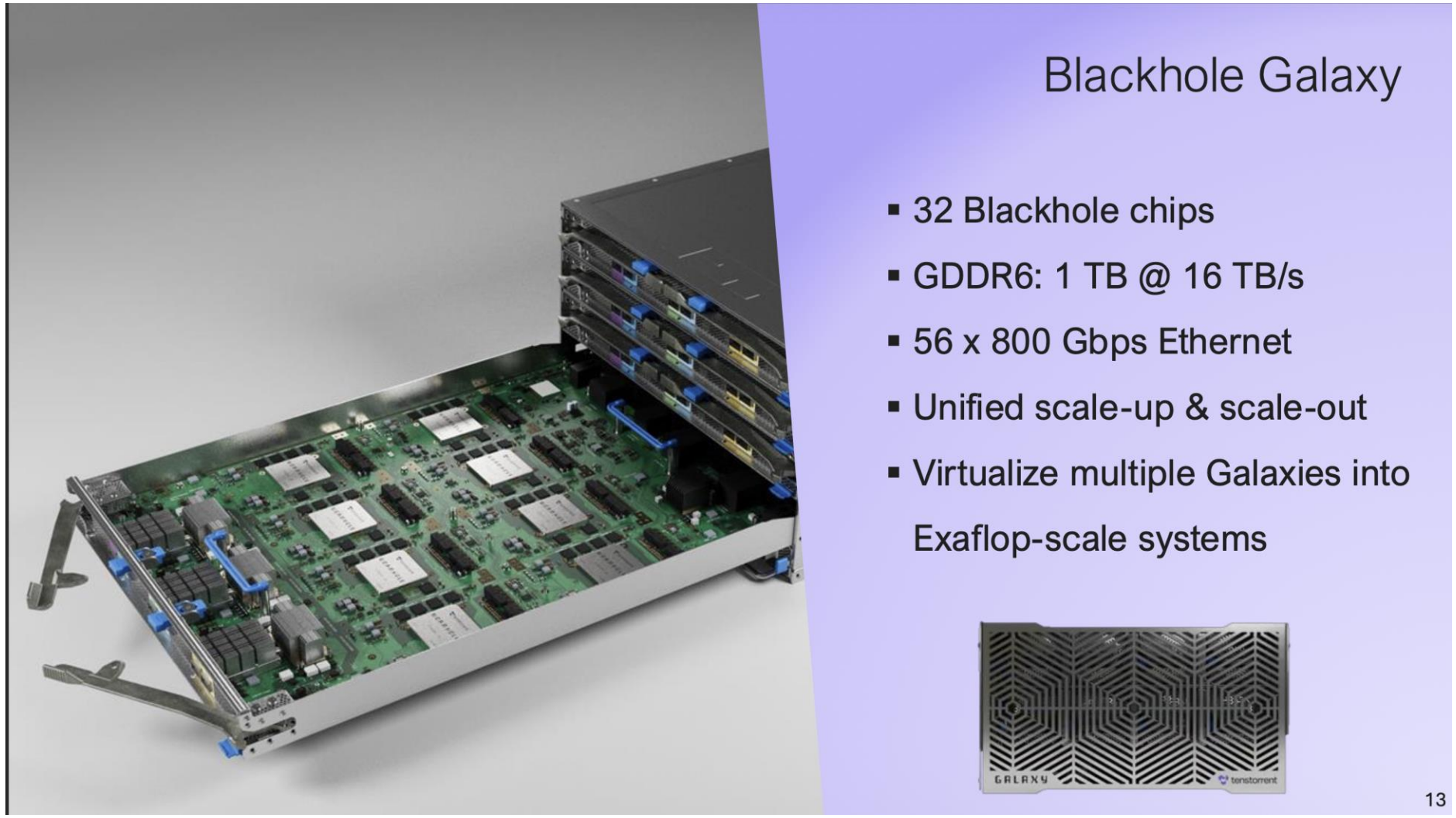

## Blackhole™

- 120 Tensix AI Processors
- 664 TFLOPs (8-bit)
- 32GB GDDR6 @ 512 GB/s
- 180MB SRAM
- 16 Risc-V Processors
- Dual 512-bit 2D Torus NOC
- PCIe Gen5 x 16
- Up to 10 x 400 Gbps Interconnect for Scale-out

12

Courtesy: Tenstorrent

# Tenstorrent

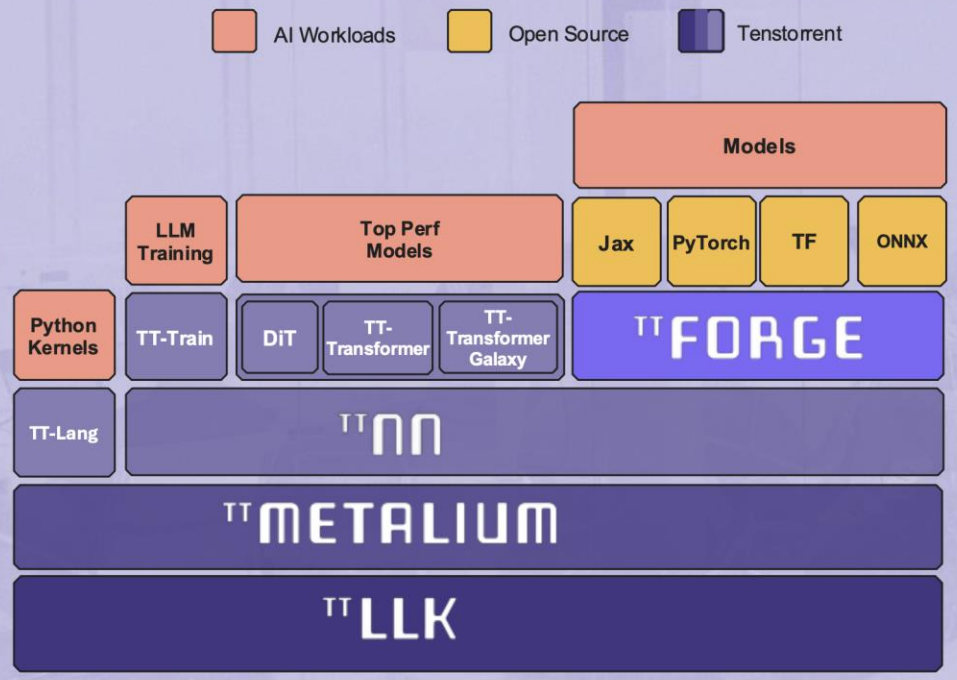



Courtesy: Tenstorrent

# Tenstorrent

## Building Block Approach

- Multiple entry points, from pytorch down to low-level kernel
- Fast growing ecosystem of partners & developers




### Third-Party Developers

| | |
|---|---|
| LEAN | Model Development |
| MAKO | AI Kernel Generation |
| epam | DX & Model Bring-up |
| ignitarium | Model Development |
| SpaceLike AI | Model Development |
| Kernelize | Triton compiler |
| inworld | In-Game AI |
| OXMIQ LABS | Cuda PyTorch Cross Compiler |
| Airev. | AI Agents |
| FlexAI | Datacenter AI Visualization |
| MOREH | AI Training |
| | ML compiler |
| | ML framework |

16

Courtesy: Tenstorrent

# ARGONNE TRAINING PROGRAM ON EXTREME-SCALE COMPUTING

Produced by Argonne National Laboratory, a U.S. Department of Energy Laboratory managed by UChicagoArgonne, LLC under contract DE-AC02-06CH11357.

Special thanks to the National Energy Research Scientific Computing Center (NERSC) and Oak Ridge Leadership Computing Facility (OLCF) for the use of their resources during the training event.